Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 10

Elementary Graph Algorithms (Part 1)

Στοιχειώδεις Αλγόριθμοι Γραφημάτων (Μέρος 1)

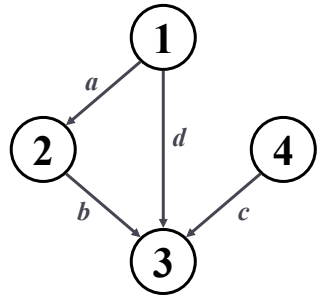
Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

1

Graphs

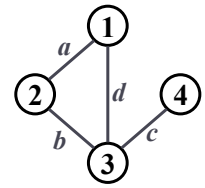
- A **graph** (**γράφημα**) $G = (V, E)$
  - V = set of **vertices** (**κόμβοι** ή **κορυφές**)
  - E = set of **edges** (**ακμές**) = subset of $V \times V$
  - Thus, $|E| = O(|V|^2)$



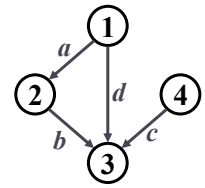
2

Graph Variations

- In an **undirected graph** (**μη κατευθυνόμενο γράφημα**):
  - Edge (u, v) = Edge (v, u)
  - No self-loops

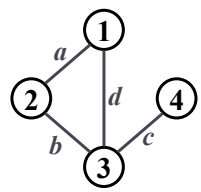


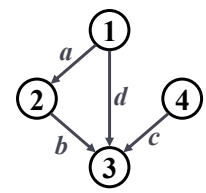
- In a **directed graph** (**κατευθυνόμενο γράφημα**) :
  - Edge (u, v) goes from vertex u to vertex v, notated $u \rightarrow v$



Graph Properties

- Graph $G = (V, E)$
  - A vertex u is **adjacent** (**γειτνιάζει**) to a vertex v iff $(u, v) \in E$
  - An edge (u, v) is **incident** (**πρόσκειται**) on vertices u and v
  - **Degree** (**βαθμός**) of a vertex is the number of edges incident on it
    - In a directed graph, the **out-degree** (**βαθμός εξόδου**) of a vertex is the number of edges leaving it, and the **in-degree** (**βαθμός εισόδου**) is the number of edges entering it
    - A vertex whose degree is 0 is **isolated** (**απομονωμένη**)





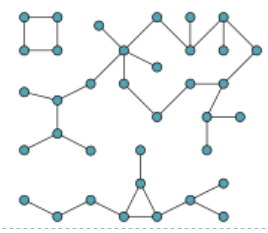
Graph Properties

- (cont'd)
  - A **walk** (**περίπατος**) is a sequence of vertices $v_0, v_1, \ldots, v_k \in V$ such that $(v_{i-1}, v_i) \in E$, for all $1 \leq i \leq k$
  - A **path** (**μονοπάτι**) is a walk where all vertices are distinct
  - A **cycle** (**κύκλος**) is a walk where $v_0 = v_k$ and all other vertices are distinct
    - A graph with no cycles is **acyclic** (**άκυκλο**)



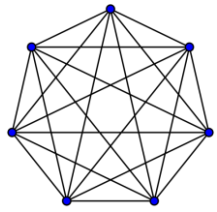
Graph Properties

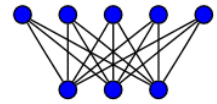
- (cont'd)
  - An undirected graph is **connected** (**συνδεδεμένο**) if every pair of vertices is connected by a path
    - The **connected components** (**συνδεδεμένες συνιστώσες**) of a graph are the equivalence classes of vertices under the "is reachable from" relation
  - A directed graph is **strongly connected** (**ισχυρά συνδεδεμένο**) if every two vertices are reachable from each other

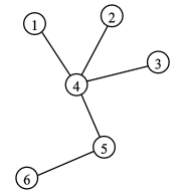


More Graph Variations

- A **complete graph** (**πλήρες γράφημα**) is an undirected graph in which every pair of vertices is adjacent
- A **bipartite graph** (**διμερές γράφημα**) is an undirected graph $G = (V, E)$ in which V can be partitioned into two sets V_1 and V_2 such that $(u, v) \in E$ implies either $u \in V_1$ and $v \in V_2$ or $u \in V_2$ and $v \in V_1$
- A **tree** (**δέντρο**) is a connected, acyclic, undirected graph
- A **forest** (**δάσος**) is an acyclic, undirected graph

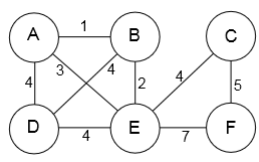


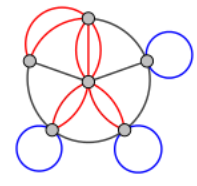




More Graph Variations

- A **weighted graph** (**εμβαρών γράφημα**) associates weights with either the edges or the vertices
  - E.g., a road map: edges might be weighted with road distance
- A **multigraph** (**πολυγράφημα**) allows multiple edges between the same vertices
  - E.g., the call graph in a program (a function can get called from multiple points in another function)





Graphs

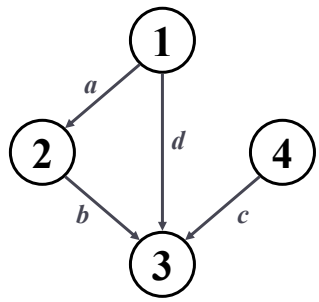
- We will typically express running times in terms of $|E|$ and $|V|$ (often dropping the $|$'s)
  - The number of vertices, $|V|$, is called **order** (**τάξη**) of G
  - The number of edges, $|E|$, is called **size** (**μέγεθος**) of G
- If $|E| \approx |V|^2$ the graph is **dense** (**πυκνό γράφημα**)
- If $|E| \approx |V|$ the graph is **sparse** (**αραιό γράφημα**)
- If you know you are dealing with dense or sparse graphs, different data structures may make sense

Representing Graphs

- Assume $V = \{1, 2, \ldots, n\}$
- An **adjacency matrix** (**πίνακας γειτνίασης**) represents the graph as a $n \times n$ matrix A:
  - $A[i, j]$ = 1 if edge $(i, j) \in E$ (or weight of edge)
    = 0 if edge $(i, j) \notin E$

Graphs: Adjacency Matrix

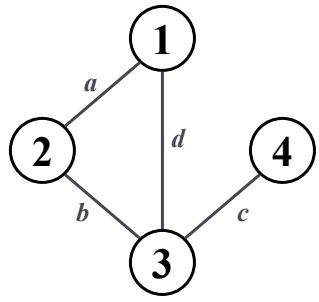
- Example:



A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

Graphs: Adjacency Matrix

- Example:



A	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	0	0	1	0

Graphs: Adjacency Matrix

- How much storage does the adjacency matrix require?
- A: $O(V^2)$

- What is the minimum amount of storage needed by an adjacency matrix representation of an undirected graph with 4 vertices?
- A: 6 bits
  - Undirected graph → matrix is symmetric
  - No self-loops → don't need diagonal

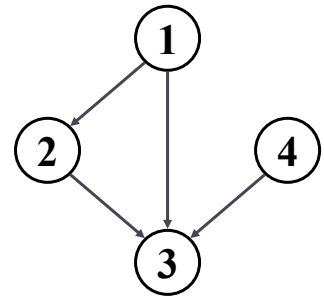
Graphs: Adjacency Matrix

- The adjacency matrix is a dense representation
  - Usually too much storage for large graphs
  - But can be very efficient for small graphs

- Most large interesting graphs are sparse
  - For this reason the *adjacency list* is often a more appropriate representation

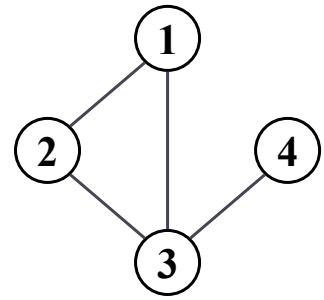
Graphs: Adjacency List

- **Adjacency list** (**κατάλογος γειτνίασης**): for each vertex $v \in V$, store a list of vertices adjacent to v
- Example:
  - Adj[1] = {2,3}
  - Adj[2] = {3}
  - Adj[3] = {}
  - Adj[4] = {3}
- Variation: can also keep a list of edges coming *into* vertex



Graphs: Adjacency List

- **Adjacency list** (**κατάλογος γειτνίασης**): for each vertex $v \in V$, store a list of vertices adjacent to v
- Example:
  - Adj[1] = {2,3}
  - Adj[2] = {1,3}
  - Adj[3] = {1,2,4}
  - Adj[4] = {3}



Graphs: Adjacency List

- How much storage is required?
  - The *degree* of a vertex v = # incident edges
    - Directed graphs have in-degree, out-degree
  - For directed graphs, # of items in adjacency lists is
    $$\Sigma \text{ out-degree}(v) = |E|$$
    takes $\Theta(V + E)$ storage (*Why?*)
  - For undirected graphs, # items in adjacency lists is
    $$\Sigma \text{ degree } (v) = 2|E|$$
    also $\Theta(V + E)$ storage
- So: Adjacency lists take $\Theta(V + E)$ storage
- What is the main disadvantage of adjacency lists?
  - To search for edge (u, v) you need to scan Adj$[u]$

Graph Searching

- Given: a graph $G = (V, E)$, directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a *tree* on the graph
- **Idea**
  - Pick a vertex as the root
  - Choose certain edges to produce a tree
  - Note: might also build a *forest*

Breadth-First Search (Οριζόντια Διερεύνηση)

- "Explore" a graph, turning it into a **tree**
  - One vertex at a time
  - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
  - Pick a ***source vertex*** (*αφετηριακή κορυφή*) to be the root
  - Find ("discover") its children, then their children, etc.

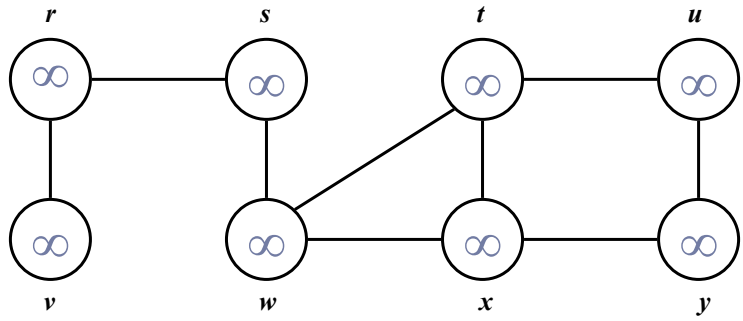
Breadth-First Search (Οριζόντια Διερεύνηση)

- Associate vertex "colors" to guide the algorithm
  - **White vertices** have not been discovered
    - All vertices start out white
  - **Grey vertices** are discovered but not fully explored
    - They may be adjacent to white vertices
  - **Black vertices** are discovered and fully explored
    - They are adjacent only to black and grey vertices
- Explore vertices by scanning adjacency list of grey vertices

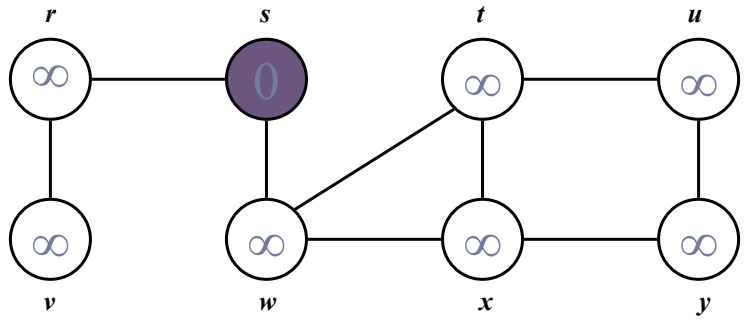
Breadth-First Search: Pseudocode

```
BFS(G, s) {
    initialize vertices;
    Q = {s};        // Q is a queue; initialize to s
    while (Q not empty) {
        u = Dequeue(Q);
        for each v ∈ G.Adj[u] {
            if (v.color == WHITE)
                v.color = GREY;
                v.d = u.d + 1;
                v.p = u;
                Enqueue(Q, v);
        }
        u.color = BLACK;
    }
}
```


Breadth-First Search: Example

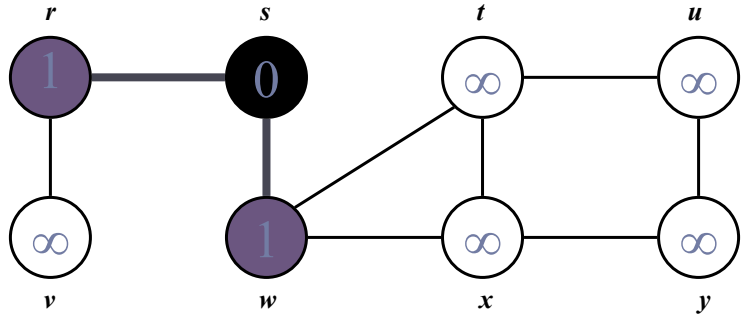


Breadth-First Search: Example



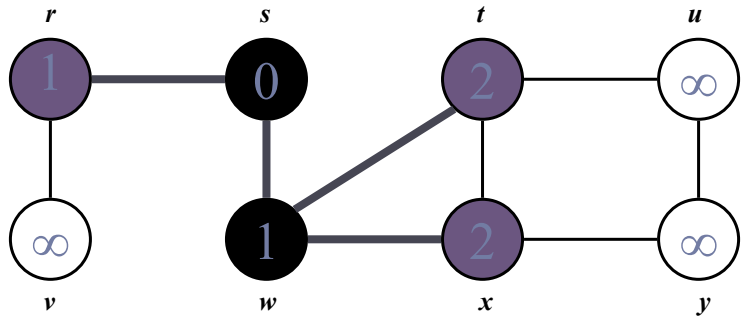
Q: | *s* |

Breadth-First Search: Example



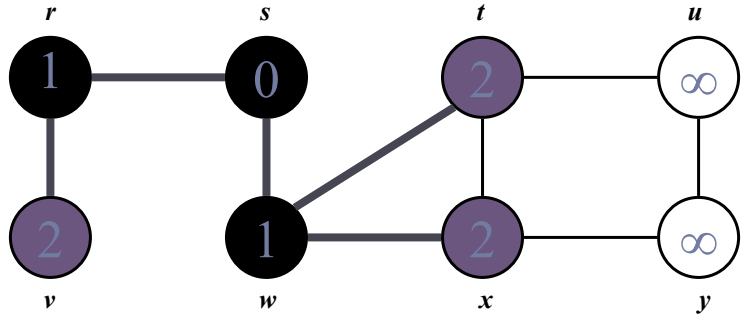
Q: | *w* | *r* |

Breadth-First Search: Example



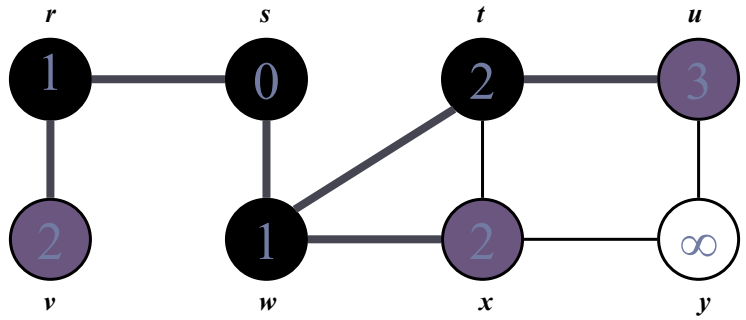
Q: | r | t | x |

Breadth-First Search: Example



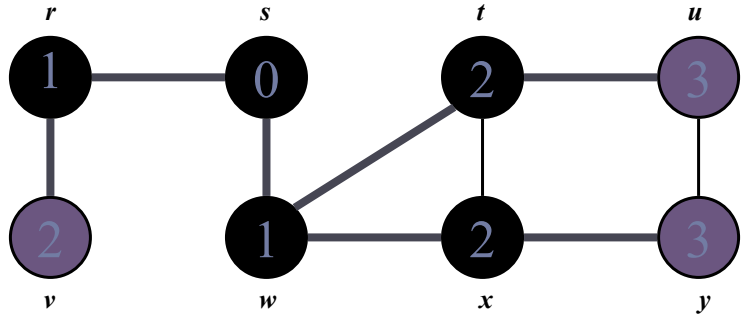
Q: | t | x | v |

Breadth-First Search: Example



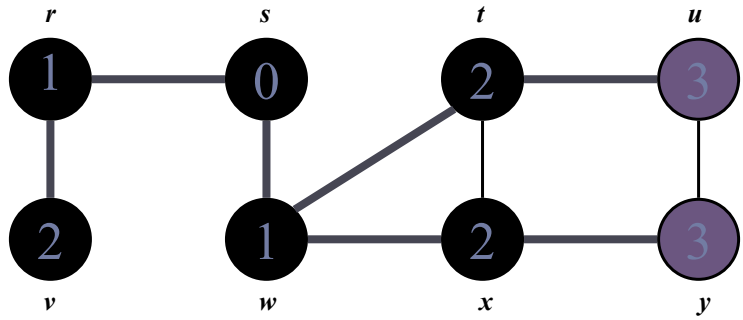
$Q:$ | x | v | u |

Breadth-First Search: Example



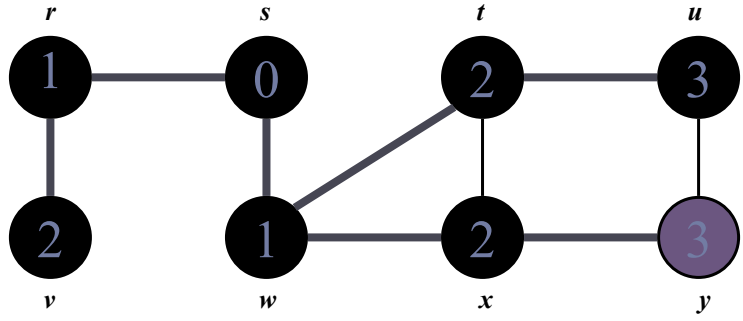
$Q:$ | v | u | y |

Breadth-First Search: Example



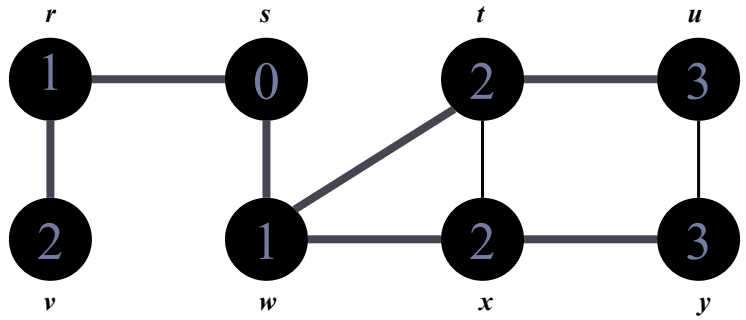
$Q:$ | u | y |

Breadth-First Search: Example



$Q:$ | y |

Breadth-First Search: Example



Q: Ø

Breadth-First Search: Pseudocode

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = Dequeue(Q);
        for each v ∈ G.Adj[u] {
            if (v.color == WHITE)
                v.color = GREY;
                v.d = u.d + 1;
                v.p = u;
                Enqueue(Q, v);
        }
        u.color = BLACK;
    }
}
```

What does **`v->d`** *represent?*

What does **`v->p`** *represent?*

Breadth-First Search: Code Analysis

```
BFS(G, s) {
    initialize vertices;
    Q = {s};
    while (Q not empty) {
        u = Dequeue(Q);
        for each v ∈ G.Adj[u] {
            if (v.color == WHITE)
                v.color = GREY;
                v.d = u.d + 1;
                v.p = u;
                Enqueue(Q, v);
        }
        u.color = BLACK;
    }
}
```

Touch every vertex: O(**V**)

u = every vertex, but only once (*Why?*)

v = every vertex that appears in some other vertex's adjacency list

What will be the running time? $\Theta(V + E)$

What will be the storage cost in addition to storing the tree? $O(V)$

Breadth-First Search: Properties

- BFS calculates the **shortest-path distance** (**απόσταση βραχύτατης διαδρομής**) to the source node
  - Shortest-path distance $\delta(s, v)$ = minimum number of edges from s to v, or ∞ if v not reachable from s
- BFS builds **breadth-first tree** (**οριζόντιο δένδρο**), in which paths to root represent shortest paths in G
  - Thus, can use BFS to calculate shortest path from one vertex to another in $O(V + E)$ time

```
PrintPath(G, s, v)
   if v == s
      print s
   else if v.p == NULL
      print "no path exists"
   else
      PrintPath(G, s, v.p)
      print v
```